

Séance 6 – Polymorphisme dynamique

Université Côte d'Azur – LJAD

Le type dynamique existe et est conservé
par une référence de base.

Mais rien, pour l'instant, ne l'utilise pour
choisir quel calcul exécuter.



Comment faire porter le choix au moment de
l'exécution ?

...mais l'appel non virtuel ne l'utilise pas

```
struct Element {  
    std::string label() const { return "Element"; }  
};  
  
struct Triangle : Element {  
    std::string label() const { return "Triangle"; }  
};  
  
void print(const Element& e) {  
    std::cout << e.label();  
}
```

Objet réel

Triangle

Type de l'accès

Element

Fonction appelée

Element::label

Un Triangle arrive bien dans print, mais l'expression `e` reste statiquement un Element.

À retenir. Nous voulons maintenant qu'un même appel produise un comportement différent selon le type dynamique.

Deux fonctions presque identiques

Version pour un triangle

```
void report(const Triangle& t) {  
    std::cout << "measure = "  
                << t.measure() << '\n';  
}
```

Version pour un quadrilatère

```
void report(const Quadrilateral& q) {  
    std::cout << "measure = "  
                << q.measure() << '\n';  
}
```

Le corps est identique. Seul le type du paramètre change.

Question. Quelle information propre au type concret le corps utilise-t-il réellement ?

Chercher la plus petite interface commune

Besoin réel de `report`

pouvoir appeler

`measure()` `const`

Ni le nombre de sommets, ni le stockage interne, ni le nom exact du type ne sont nécessaires à cette fonction.

```
class Element {  
public:  
    double measure() const;  
};  
  
void report(const Element& e) {  
    std::cout << "measure = "  
                << e.measure() << '\n';  
}
```

Nous cherchons une **interface minimale**, pas une taxonomie exhaustive du maillage.

Vigilance. Avec cette déclaration ordinaire, l'appel suivrait encore le type statique. Il manque le mécanisme de choix.

virtual : le choix suit le type dynamique

```
struct Element {  
    virtual double measure() const { return 0.0; }  
};  
  
struct Triangle : Element {  
    double measure() const { return area_; }  
    double area_{2.5};  
};  
  
void report(const Element& e) {  
    std::cout << e.measure();  
}  
  
Triangle t;  
report(t);                // affiche 2.5
```

Type statique de e

Element

Type dynamique

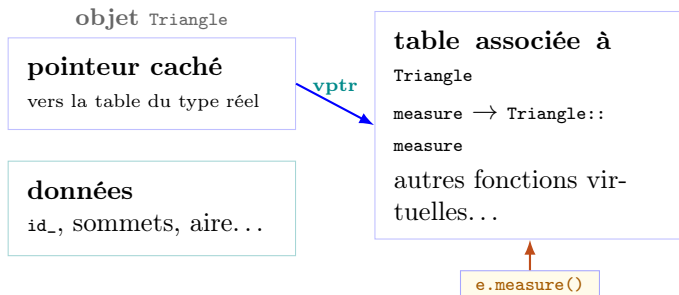
Triangle

Fonction choisie

Triangle::measure

À retenir. Le mot-clé `virtual` demande que la sélection de la fonction tienne compte du type dynamique.

Un modèle mental du dispatch dynamique



Dispatch dynamique

L'appel passe conceptuellement par une table propre au type réel, au lieu d'être fixé directement.

Précision

`vp_ptr/vtable` est une implémentation très courante, mais la norme C++ n'impose ni ce nom ni cette disposition mémoire.

Fonction virtuelle pure : = 0

```
struct Element {  
    virtual double measure() const = 0;  
};  
  
struct Triangle : Element {  
    double measure() const { return 2.5; }  
};  
  
Element e;    // erreur : classe abstraite  
Triangle t;   // possible : measure() est fourni
```

= 0 **signifie**

La base n'offre pas de comportement par défaut pour cette opération.

Conséquence

Element devient abstraite : on ne peut pas créer directement un objet de ce type.

Le zéro ne veut pas dire que la fonction renvoie zéro. Il marque une **obligation de redéfinition**.

Une classe abstraite peut avoir du code et de l'état

```
class Element {  
public:  
    int id() const { return id_; }           // code  
        concret  
    virtual double measure() const = 0;      //  
        contrat abstrait  
  
protected:  
    explicit Element(int id) : id_{id} {}    //  
        constructeur  
  
private:  
    int id_;                                // etat  
};
```

- ✓ données membres
- ✓ constructeur
- ✓ fonctions concrètes
- ✓ références et pointeurs

✗ instantiation directe
de `Element`

Une dérivée appelle le constructeur de la base, réutilise ses fonctions concrètes et fournit les opérations pures manquantes.

À retenir. « Abstraite » ne signifie pas « vide » ni « sans code ».

Le bug silencieux : une signature presque identique

```
struct Element {  
    virtual std::string label() const {  
        return "Element";  
    }  
};  
  
struct Triangle : Element {  
    std::string label() {           // const oublie  
        return "Triangle";  
    }  
};  
  
Triangle t;  
const Element& e = t;  
std::cout << e.label();           // affiche "Element"
```

Base

label() const

Dérivée

label()

signature différente

La fonction de la dérivée **masque** le nom de la base, mais ne la redéfinit pas. Le code compile.

Vigilance. Un `const` oublié suffit à empêcher la redéfinition attendue.

```
struct Element {  
    virtual std::string label() const;  
};  
  
struct Triangle : Element {  
    std::string label() override;  
    // ~ erreur  
    // "marked override, but does not override"  
};
```

Le compilateur signale immédiatement que la signature ne correspond à aucune fonction virtuelle de la base.

Ce que vérifie `override`
nom, paramètres, type de retour compatible, `const`, qualificateurs de référence...

Ce qu'il ne fait pas

Il ne rend pas la fonction virtuelle : elle l'est déjà par héritage.

À retenir. Écrire `override` sur chaque redéfinition transforme un bug silencieux en erreur de compilation.

Détruire via la base sans destructeur virtuel

```
struct Element {  
    virtual double measure() const = 0;  
    ~Element() = default;           // non  
    virtuel  
};  
  
struct Triangle : Element {  
    std::vector<double> cache;  
    double measure() const { return 2.5; }  
};  
  
Element* p = new Triangle;  
delete p;                               //  
    comportement indefini
```

Le problème

Le type réel est `Triangle`, mais la destruction est lancée à travers un pointeur de base dont le destructeur n'est pas virtuel.

Le programme n'a **aucun résultat défini**. Une ressource propre à la dérivée peut ne pas être libérée correctement.

```
g++ -std=c++20 -g -fsanitize=address,undefined demo.cpp
```

Le correctif : un destructeur virtuel

```
struct Element {  
    virtual double measure() const = 0;  
    virtual ~Element() = default;  
};  
  
Element* p = new Triangle;  
delete p;           // destruction  
                    complete
```

Même correction pour `std::unique_ptr<Element>` : sa destruction doit pouvoir remonter jusqu'au type réel.

Ordre garanti

1. partie dérivée
destructeur de `Triangle`



2. sous-objet de
base
destructeur de `Element`

À retenir. Règle pratique : une base destinée à être détruite polymorphiquement possède un destructeur public virtuel.

Slicing : la copie qui efface le type dérivé

```
struct Element {                                // base concrete
    ici
    virtual std::string label() const {
        return "Element";
    }
    virtual ~Element() = default;
};

struct Triangle : Element {
    std::string label() const override {
        return "Triangle";
    }
};

void show(Element e) {                          // passage par
    valeur
    std::cout << e.label();
}

show(Triangle{});                             // affiche "
    Element"
```

Avant l'appel

objet réel : Triangle

Dans show

nouvelle copie : Element seulement

Une base abstraite interdit cette copie précise. Une base concrète, elle, peut laisser passer le bug silencieusement.

Éviter le slicing : référence ou pointeur

À éviter pour un type polymorphe

```
void show(Element e);           // copie
std::vector<Element> elements;  //
    copies de base
```

✗ Le type dérivé est perdu lors de la copie. Le dispatch dynamique ne peut plus le retrouver.

À utiliser

```
void show(const Element& e);    // pas
    de copie
Element* p = &triangle;

std::vector<std::unique_ptr<Element>>
    elements;
```

✓ Référence ou pointeur conserve l'objet complet et son type dynamique.

À retenir. Pour un type polymorphe : passage par référence ; propriété éventuelle par pointeur intelligent.

Quand le polymorphisme dynamique est le bon outil

Type choisi à l'exécution
un fichier de maillage indique triangle, quadrilatère ou tétraèdre.

Collection hétérogène
plusieurs types concrets derrière une interface commune.

Comportement extensible
solveur, stratégie ou greffon sélectionné sans réécrire le client.

```
std::vector<std::unique_ptr<Element>> elements;  
  
elements.push_back(  
    std::make_unique<Triangle>(/* ... */));  
  
elements.push_back(  
    std::make_unique<Quadrilateral>(/* ... */));  
  
for (const auto& e : elements) {  
    std::cout << e->measure() << '\n';  
}
```

La gestion détaillée de la propriété sera reprise avec les pointeurs intelligents.

Le coût existe, mais il faut le nommer correctement

Coûts possibles

Appel indirect

une consultation de table peut remplacer un appel direct.

Taille de l'objet

une implémentation courante ajoute un pointeur caché.

Propriété

une collection hétérogène demande souvent des pointeurs.

Ce qui n'est pas automatiquement vrai

✓ `virtual` n'impose pas l'allocation sur le tas.

✓ Le compilateur peut parfois dévirtualiser et réinline.

✗ Un simple `Point2D` n'a pas besoin d'une hiérarchie : deux `double` forment un bon type valeur.

Quatre mécanismes à retenir

virtual

L'appel dépend du type dynamique de l'objet désigné.

= 0

La base impose une opération et devient abstraite.

override

Le compilateur vérifie la redéfinition exacte.

destructeur virtuel

La destruction via la base atteint le type réel.

Référence de base + objet dérivé + appel virtuel = polymorphisme dynamique

Frontière suivante : choix dynamique ou choix statique ?

Choix à l'exécution

`virtual`

famille ouverte, types hétérogènes, indirection possible

Choix à la compilation

templates

types connus lors de la compilation, spécialisation statique, autre compromis

Vigilance finale Un objet polymorphe se manipule par référence ou pointeur : la copie par valeur peut produire du slicing.

La question suivante sera aussi celle de la propriété : qui détruit ces objets ? `std::unique_ptr` complètera le tableau.